Class / Function Name	# No.	Test Description	Sample Input	Expected Result	Actual Result	P/F
Class: IngredientSlot						
IngredientSlot	1	Constructs slot with valid name and standard quantity	"STRAWBERRY", 5	Object created with itemName="STRAWBERRY", quantity=5	Object created with itemName="STRAWBERRY", quantity=5	P
IngredientSlot	2	Constructs slot with zero quantity (Edge Case)	"APPLE", 0	Object created with itemName="APPLE", quantity=0	Object created with itemName="APPLE", quantity=0	P
IngredientSlot	3	Constructs slot with negative quantity (Invalid)	"BANANA", -3	Throws IllegalArgumentException or defaults quantity to 0	Throws IllegalArgumentException or defaults quantity to 0	P
getItemName	1	Retrieves name from a populated slot	None (Slot has "STRAWBERRY")	Returns "STRAWBERRY"	Returns "STRAWBERRY"	P
getItemName	2	Retrieves name from an empty slot	None (Slot is empty)	Returns "Empty" or null	Returns "Empty" or null	P
getItemName	3	Retrieves name with special characters/spaces	None (Slot has "MAGIC BEANS")	Returns "MAGIC BEANS" accurately	Returns "MAGIC BEANS" accurately	P
getQuantity	1	Retrieves standard available quantity	None (Qty is 5)	Returns 5	Returns 5	P
getQuantity	2	Retrieves quantity when slot is empty (Edge Case)	None (Qty is 0)	Returns 0	Returns 0	P
getQuantity	3	Retrieves quantity after maximum items added	None (Qty is 99)	Returns 99	Returns 99	P
emptySlot	1	Clears a slot that currently contains items	None	itemName="Empty", quantity=0	itemName="Empty", quantity=0	P
emptySlot	2	Clears a slot that is already empty (Edge Case)	None	Remains itemName="Empty", quantity=0	Remains itemName="Empty", quantity=0	P
emptySlot	3	Verify quantity cannot be retrieved after emptying	None	getQuantity() returns 0	getQuantity() returns 0	P
Class: Market						
Market	1	Constructs Market and initializes slot array	None	Array size 8 initialized	Array size 8 initialized	P
Market	2	Verify array size boundaries	None	Accessing index 8 throws exception	Accessing index 8 throws exception	P
Market	3	Verify slots are populated upon instantiation	None	All 8 slots contain non-null objects	All 8 slots contain non-null objects	P
refreshMarket	1	Populates randomly with max 1 Cauldron	None	Slots filled; count of "CAULDRON" <= 1	Slots filled; count of "CAULDRON" <= 1	P
refreshMarket	2	Refreshes multiple times to ensure true randomness	None	Subsequent calls yield different item arrays	Subsequent calls yield different item arrays	P
refreshMarket	3	Verify Cauldron does not appear if player already owns max	None (Player owns Cauldron)	Slots filled; count of "CAULDRON" = 0	Slots filled; count of "CAULDRON" = 0	P
getSlot	1	Retrieves slot at a valid middle index	3	Returns IngredientSlot object at index 3	Returns IngredientSlot object at index 3	P
getSlot	2	Retrieves slot at lowest/highest boundary (Edge Case)	0 and 7	Returns valid IngredientSlot objects	Returns valid IngredientSlot objects	P
getSlot	3	Retrieves slot at out-of-bounds index (Invalid)	8 or -1	Throws IndexOutOfBoundsException	Throws IndexOutOfBoundsException	P
Class: Inventory						
Inventory	1	Constructs an inventory with 3 default cauldrons	None	3 Usable Cauldron objects initialized	3 Usable Cauldron objects initialized	P
Inventory	2	Verifies fruit list is initially empty	None	Fruit list count is 0	Fruit list count is 0	P
Inventory	3	Verifies base list is initially empty	None	Base list count is 0	Base list count is 0	P
addFruit	1	Adds a new fruit to the inventory list	"MANGO", 2	Ingredient "MANGO" added with qty 2	Ingredient "MANGO" added with qty 2	P
addFruit	2	Stacks quantity for an existing fruit	"MANGO", 3	"MANGO" quantity increases to 5	"MANGO" quantity increases to 5	P
addFruit	3	Fails to add fruit with a negative/zero quantity	"MANGO", -1	Request rejected, quantity unchanged	Request rejected, quantity unchanged	P
removeFruit	1	Deducts fruit quantity if sufficient amount exists	"MANGO", 2	Returns true, quantity decreases	Returns true, quantity decreases	P
removeFruit	2	Fails to deduct if quantity is insufficient	"MANGO", 10	Returns false, quantity unchanged	Returns false, quantity unchanged	P
removeFruit	3	Deducts exact total quantity (reaches 0)	"MANGO", 2	Returns true, quantity becomes 0	Returns true, quantity becomes 0	P
addBase	1	Adds a new base to the inventory list	"MILK BASE", 1	Ingredient "MILK BASE" added with qty 1	Ingredient "MILK BASE" added with qty 1	P
addBase	2	Stacks quantity for an existing base	"MILK BASE", 2	"MILK BASE" quantity increases to 3	"MILK BASE" quantity increases to 3	P
addBase	3	Fails to add base with a negative/zero quantity	"MILK BASE", 0	Request rejected, quantity unchanged	Request rejected, quantity unchanged	P
removeBase	1	Deducts base quantity if sufficient amount exists	"MILK BASE", 1	Returns true, quantity decreases	Returns true, quantity decreases	P
removeBase	2	Fails to deduct if quantity is insufficient	"MILK BASE", 5	Returns false, quantity unchanged	Returns false, quantity unchanged	P
removeBase	3	Deducts exact total quantity (reaches 0)	"MILK BASE", 1	Returns true, quantity becomes 0	Returns true, quantity becomes 0	P

Class / Function Name	# No.	Test Description	Sample Input	Expected Result	Actual Result	P/F
checkIngredientAvailability	1	Checks if sufficient qty of existing ingredient exists	"MANGO", 1	Returns true	Returns true	P
checkIngredientAvailability	2	Checks if requested qty exceeds current stock	"MANGO", 100	Returns false	Returns false	P
checkIngredientAvailability	3	Checks availability of a non-existent ingredient	"STRAWBERRY", 1	Returns false	Returns false	P
getUsableCauldronCount	1	Calculates number of usable cauldrons on init	None	Returns 3 (default start)	Returns 3 (default start)	P
getUsableCauldronCount	2	Calculates count after a new cauldron is added	None	Returns 4 (after addCauldron)	Returns 4 (after addCauldron)	P
getUsableCauldronCount	3	Calculates count after a cauldron is ruined	None	Returns 2 (after ruinOneCauldron)	Returns 2 (after ruinOneCauldron)	P
getUnusableCauldronCount	1	Calculates number of ruined cauldrons on init	None	Returns 0 (default start)	Returns 0 (default start)	P
getUnusableCauldronCount	2	Calculates count after one cauldron is ruined	None	Returns 1	Returns 1	P
getUnusableCauldronCount	3	Calculates count after ruined cauldron is blessed	None	Returns 0	Returns 0	P
ruinOneCauldron	1	Changes state of one usable cauldron to ruined	None	Usable count -1, Unusable count +1	Usable count -1, Unusable count +1	P
ruinOneCauldron	2	Attempts to ruin when no usable cauldrons exist	None	Returns error/false, counts unchanged	Returns error/false, counts unchanged	P
ruinOneCauldron	3	Multiple ruins stack properly	Run 3x	Usable -3, Unusable +3	Usable -3, Unusable +3	P
blessOneCauldron	1	Restores one ruined cauldron back to usable	None	Usable count +1, Unusable count -1	Usable count +1, Unusable count -1	P
blessOneCauldron	2	Attempts to bless when no ruined cauldrons exist	None	Returns error/false, counts unchanged	Returns error/false, counts unchanged	P
blessOneCauldron	3	Multiple blessings stack properly	Run 2x	Usable +2, Unusable -2	Usable +2, Unusable -2	P
addCauldron	1	Adds a new usable cauldron to the inventory	None	Usable cauldron count increases by 1	Usable cauldron count increases by 1	P
addCauldron	2	Adds multiple cauldrons successively	Add 5x	Usable cauldron count increases by 5	Usable cauldron count increases by 5	P
addCauldron	3	Verifies ruined cauldron count is unaffected	None	Unusable count remains unchanged	Unusable count remains unchanged	P
displayInventory	1	Prints current state of a populated inventory	None	Console printout matching lists	Console printout matching lists	P
displayInventory	2	Prints gracefully for completely empty inventory	None	Console printout shows 0 for lists	Console printout shows 0 for lists	P
displayInventory	3	Verifies formatting matches expected template	None	Output matches exact string template	Output matches exact string template	P
exportFruitData	1	Compiles fruit inventory to a formatted string	None	Formatted string of multiple fruits	Formatted string of multiple fruits	P
exportFruitData	2	Returns appropriate output when no fruits exist	None	Empty string or "Empty" message	Empty string or "Empty" message	P
exportFruitData	3	Verifies correct ordering/formatting in string	None	String order matches insertion/alpha	String order matches insertion/alpha	P
Class: Ingredient						
Ingredient	1	Constructs with name and initial quantity	BANANA, 3	Object created with name="BANANA", qty=3	Object created with name="BANANA", qty=3	P
Ingredient	2	Constructs with name and zero quantity	APPLE, 0	Object created with name="APPLE", qty=0	Object created with name="APPLE", qty=0	P
Ingredient	3	Constructs with single character name	X, 10	Object created with name="X", qty=10	Object created with name="X", qty=10	P
getName	1	Retrieves standard ingredient name	None	BANANA	BANANA	P
getName	2	Retrieves ingredient name with spaces	None	RED APPLE	RED APPLE	P
getName	3	Retrieves lowercase ingredient name	None	orange	orange	P
getQuantity	1	Retrieves standard positive quantity	None	3	3	P
getQuantity	2	Retrieves zero quantity	None	0	0	P
getQuantity	3	Retrieves large quantity	None	999	999	P
addQuantity	1	Increases quantity by positive amount	2	Quantity becomes 5	Quantity becomes 5	P
addQuantity	2	Ignores invalid negative amount	-1	Quantity remains unchanged	Quantity remains unchanged	P
addQuantity	3	Adds zero to current quantity	0	Quantity remains unchanged	Quantity remains unchanged	P
deductQuantity	1	Decreases quantity by a valid amount	1	Returns true, qty becomes 2	Returns true, qty becomes 2	P
deductQuantity	2	Fails to deduct if amount > current stock	10	Returns false, qty unchanged	Returns false, qty unchanged	P

Class / Function Name	# No.	Test Description	Sample Input	Expected Result	Actual Result	P/F
deductQuantity	3	Deducts exactly the current stock to zero	3	Returns true, qty becomes 0	Returns true, qty becomes 0	P
Class: Recipe						
Recipe	1	Constructs with standard ID, name, base, and price	1, "Health Potion", "MILK BASE", 150	Object created successfully	Object created successfully	P
Recipe	2	Constructs with zero price	2, "Free Potion", "WATER BASE", 0	Object created successfully	Object created successfully	P
Recipe	3	Constructs with empty name	3, "", "SYRUP BASE", 50	Object created successfully	Object created successfully	P
addRequiredFruit	1	Appends first fruit to empty list	ORANGE	ORANGE added to requiredFruits	ORANGE added to requiredFruits	P
addRequiredFruit	2	Appends second fruit to existing list	APPLE	APPLE added to requiredFruits	APPLE added to requiredFruits	P
addRequiredFruit	3	Appends duplicate fruit to list	ORANGE	ORANGE added to requiredFruits	ORANGE added to requiredFruits	P
getId	1	Retrieves standard ID	None	1	1	P
getId	2	Retrieves large ID	None	99	99	P
getId	3	Retrieves zero ID	None	0	0	P
getBaseName	1	Retrieves standard base name	None	MILK BASE	MILK BASE	P
getBaseName	2	Retrieves alternate base name	None	WATER BASE	WATER BASE	P
getBaseName	3	Retrieves base name with hyphen	None	SODA-BASE	SODA-BASE	P
getName	1	Retrieves standard display name	None	Health Potion	Health Potion	P
getName	2	Retrieves alternate display name	None	Mana Potion	Mana Potion	P
getName	3	Retrieves short display name	None	X	X	P
getRequiredFruits	1	Retrieves empty fruit list	None	Empty ArrayList	Empty ArrayList	P
getRequiredFruits	2	Retrieves list with one fruit	None	ArrayList containing "ORANGE"	ArrayList containing "ORANGE"	P
getRequiredFruits	3	Retrieves list with multiple fruits	None	ArrayList containing "ORANGE", "APPLE"	ArrayList containing "ORANGE", "APPLE"	P
getPrice	1	Retrieves standard base price	None	150	150	P
getPrice	2	Retrieves zero base price	None	0	0	P
getPrice	3	Retrieves large base price	None	9999	9999	P
matchesIngredients	1	Validates perfectly matching base and fruits	MILK BASE, ["ORANGE"]	Returns true	Returns true	P
matchesIngredients	2	Rejects mismatched base ingredient	SYRUP BASE, ["ORANGE"]	Returns false	Returns false	P
matchesIngredients	3	Rejects missing required fruits	MILK BASE, []	Returns false	Returns false	P
Class: Player						
Player	1	Constructs player with name and default balance	Sam-Paul	Name="Sam-Paul", Crystals=5000	Name="Sam-Paul", Crystals=5000	P
Player	2	Constructs player with single character name	X	Name="X", Crystals=5000	Name="X", Crystals=5000	P
Player	3	Constructs player with space in name	John Doe	Name="John Doe", Crystals=5000	Name="John Doe", Crystals=5000	P
getName	1	Retrieves standard player name	None	Sam-Paul	Sam-Paul	P
getName	2	Retrieves single character name	None	X	X	P
getName	3	Retrieves name with spaces	None	John Doe	John Doe	P
getCrystals	1	Retrieves default crystal balance	None	5000	5000	P
getCrystals	2	Retrieves balance after adding crystals	None	5500	5500	P
getCrystals	3	Retrieves empty balance	None	0	0	P
getInventory	1	Retrieves initial empty inventory	None	Inventory instance (empty)	Inventory instance (empty)	P
getInventory	2	Verifies inventory reference is not null	None	Valid Inventory reference	Valid Inventory reference	P
getInventory	3	Retrieves inventory after adding items	None	Inventory instance (1 item)	Inventory instance (1 item)	P

Class / Function Name	# No.	Test Description	Sample Input	Expected Result	Actual Result	P/F
getSpellbook	1	Retrieves initial empty spellbook	None	Spellbook instance (empty)	Spellbook instance (empty)	P
getSpellbook	2	Verifies spellbook reference is not null	None	Valid Spellbook reference	Valid Spellbook reference	P
getSpellbook	3	Retrieves spellbook after learning spells	None	Spellbook instance (1 spell)	Spellbook instance (1 spell)	P
addCrystals	1	Adds positive amount to crystal balance	500	Crystals become 5500	Crystals become 5500	P
addCrystals	2	Adds zero to current balance	0	Crystals remain unchanged	Crystals remain unchanged	P
addCrystals	3	Ignores invalid negative amount	-100	Crystals remain unchanged	Crystals remain unchanged	P
deductCrystals	1	Deducts valid amount from balance	2000	Returns true, crystals become 3000	Returns true, crystals become 3000	P
deductCrystals	2	Fails to deduct amount greater than balance	9999	Returns false, crystals unchanged	Returns false, crystals unchanged	P
deductCrystals	3	Deducts exact balance to zero	5000	Returns true, crystals become 0	Returns true, crystals become 0	P
Class: Spellbook						
Spellbook	1	Constructs empty unlocked recipes list	None	Empty unlockedRecipes list	Empty unlockedRecipes list	P
Spellbook	2	Verifies unlocked recipes list is not null	None	Valid ArrayList reference	Valid ArrayList reference	P
Spellbook	3	Initializes spellbook with zero total capacity used	None	Total capacity used is 0	Total capacity used is 0	P
addRecipe	1	Adds new recipe if not already unlocked	Recipe obj (ID 1)	Recipe added successfully	Recipe added successfully	P
addRecipe	2	Ignores adding an already unlocked recipe	Recipe obj (ID 1)	Duplicate ignored, size remains same	Duplicate ignored, size remains same	P
addRecipe	3	Adds multiple unique recipes	Recipe obj (ID 2, 3)	Multiple recipes added successfully	Multiple recipes added successfully	P
hasRecipe	1	Confirms an existing unlocked recipe ID	1	Returns true	Returns true	P
hasRecipe	2	Returns false for locked or missing recipe ID	99	Returns false	Returns false	P
hasRecipe	3	Returns false for invalid negative recipe ID	-1	Returns false	Returns false	P
displaySpellbook	1	Displays empty spellbook message	None	Console printout indicating empty	Console printout indicating empty	P
displaySpellbook	2	Displays single unlocked recipe	None	Console printout with 1 item	Console printout with 1 item	P
displaySpellbook	3	Displays and sorts multiple unlocked recipes by ID	None	Console printout in ascending ID order	Console printout in ascending ID order	P
exportSpellbookData	1	Compiles empty spellbook to empty string	None	Returns empty string ""	Returns empty string ""	P
exportSpellbookData	2	Compiles single unlocked ID to a formatted string	None	Formatted string "1,"	Formatted string "1,"	P
exportSpellbookData	3	Compiles multiple unlocked IDs to a formatted string	None	Formatted string "1,2,3,"	Formatted string "1,2,3,"	P
Class: GameController						
GameController	1	Constructs controller with dependencies	None	Objects instantiated successfully	Objects instantiated successfully	P
GameController	2	Initializes player state to null before start	None	Player state initialized to null	Player state initialized to null	P
GameController	3	Calls loadCompendium during initialization	None	loadCompendium executed on startup	loadCompendium executed on startup	P
loadCompendium	1	Parses valid recipes into compendium	None	recipeCompendium populated with valid recipes	recipeCompendium populated with valid recipes	P
loadCompendium	2	Handles empty recipe source gracefully	None	recipeCompendium initialized empty	recipeCompendium initialized empty	P
loadCompendium	3	Ignores malformed entries during parse	Invalid format line	Loads only valid entries	Loads only valid entries	P
startGame	1	Starts new game with valid player name	NewPlayer	Main loop begins with new Player	Main loop begins with new Player	P
startGame	2	Starts game bypassing empty name input	None	Main loop begins with default Player	Main loop begins with default Player	P
startGame	3	Loads existing player state if found	LoadSave	Main loop begins with loaded Player	Main loop begins with loaded Player	P
mainMenuLoop	1	Processes valid menu selection (Brew)	1	Mapss to brew menu	Mapss to brew menu	P
mainMenuLoop	2	Re-prompts on invalid menu selection	99	Loop continues and re-prompts	Loop continues and re-prompts	P
mainMenuLoop	3	Processes exit command	7	Calls saveGame() and exits loop	Calls saveGame() and exits loop	P
brewMenu	1	Displays available brewing modes	None	Console prints brew options	Console prints brew options	P

Class / Function Name	# No.	Test Description	Sample Input	Expected Result	Actual Result	P/F
brewMenu	2	Navigates to recipe mode on selection	1	Transitions to recipeMode logic	Transitions to recipeMode logic	P
brewMenu	3	Cancels and returns to main loop	3	Loop returns to main menu	Loop returns to main menu	P
recipeMode	1	Processes valid brew with sufficient ingredients	1/Y	Potion brewed and ingredients consumed	Potion brewed and ingredients consumed	P
recipeMode	2	Rejects brew due to insufficient ingredients	2/Y	Brew fails and stock remains unchanged	Brew fails and stock remains unchanged	P
recipeMode	3	Cancels brew operation before confirmation	1/N	Brew cancelled and returns to menu	Brew cancelled and returns to menu	P
creativeMode	1	Processes valid base and fruit combination	MILK BASE, ORANGE	Mix validated and recipe discovered	Mix validated and recipe discovered	P
creativeMode	2	Processes invalid combination	WATER BASE, DIRT	Mix fails and ruins cauldron	Mix fails and ruins cauldron	P
creativeMode	3	Rejects mixing with missing inventory	MILK BASE, APPLE	Mix fails due to missing items	Mix fails due to missing items	P
visitMarket	1	Enters market and displays options	None	Market sub-menu displayed	Market sub-menu displayed	P
visitMarket	2	Processes valid buy transaction	1/Y	Item added and crystals deducted	Item added and crystals deducted	P
visitMarket	3	Exits market sub-menu	3	Loop returns to main menu	Loop returns to main menu	P
getIngredientPrice	1	Retrieves valid purchase price for CAULDRON	CAULDRON	Returns 3000	Returns 3000	P
getIngredientPrice	2	Retrieves valid purchase price for APPLE	APPLE	Returns 100	Returns 100	P
getIngredientPrice	3	Returns error identifier for invalid item	INVALID_ITEM	Returns -1	Returns -1	P
getSellPrice	1	Retrieves valid sell price for BANANA	BANANA	Returns 50	Returns 50	P
getSellPrice	2	Retrieves valid sell price for potion	Health Potion	Returns base price plus premium	Returns base price plus premium	P
getSellPrice	3	Returns zero for unsellable item	LOCKED_RECIPE	Returns 0	Returns 0	P
claimLoginBonus	1	Grants bonus if not yet claimed	None	Adds item and sets flag to true	Adds item and sets flag to true	P
claimLoginBonus	2	Rejects bonus if already claimed today	None	Returns false and no item granted	Returns false and no item granted	P
claimLoginBonus	3	Processes bonus with full inventory	None	Grants alternative crystals	Grants alternative crystals	P
blessCauldronLogic	1	Deducts 1000 crystals to fix ruined cauldron	Y	Crystals -1000, cauldron restored	Crystals -1000, cauldron restored	P
blessCauldronLogic	2	Fails to fix if crystals < 1000	Y	Fix fails due to insufficient funds	Fix fails due to insufficient funds	P
blessCauldronLogic	3	Rejects fixing an already working cauldron	Y	Fix cancelled as cauldron is working	Fix cancelled as cauldron is working	P
saveGame	1	Saves valid player state to file	None	State written to file successfully	State written to file successfully	P
saveGame	2	Overwrites existing save data correctly	None	Previous save overwritten successfully	Previous save overwritten successfully	P
saveGame	3	Handles save operation with empty inventory	None	State written to file successfully	State written to file successfully	P
Class: Cauldron						
Cauldron	1	Constructs with a usable state by default	None	isUsable initialized to true	isUsable initialized to true	P
Cauldron	2	Verifies object reference is not null	None	Valid Cauldron instance created	Valid Cauldron instance created	P
Cauldron	3	Instantiates without runtime errors	None	Object instantiated successfully	Object instantiated successfully	P
checkUsability	1	Checks operational state on fresh cauldron	None	Returns true	Returns true	P
checkUsability	2	Checks operational state after ruinCauldron	None	Returns false	Returns false	P
checkUsability	3	Checks operational state after blessCauldron	None	Returns true	Returns true	P
ruinCauldron	1	Marks the functional cauldron as unusable	None	isUsable becomes false	isUsable becomes false	P
ruinCauldron	2	Re-applies ruin on an already ruined cauldron	None	isUsable remains false	isUsable remains false	P
ruinCauldron	3	Verifies ruin state persists across checks	None	State verified as false	State verified as false	P
blessCauldron	1	Restores broken cauldron to functional state	None	isUsable resets to true	isUsable resets to true	P
blessCauldron	2	Applies blessing to an already functional cauldron	None	isUsable remains true	isUsable remains true	P
blessCauldron	3	Verifies blessed state persists across checks	None	State verified as true	State verified as true	P